SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**K.GNANESWAR KUMAR**	**Reg.No.:**	**192572105**
Department:	**BE-CSE(AI)**	**Date:**	**01-09-2026**

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I K.GNANESWAR KUMAR(192572105)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:k.gnaneswar kumar

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Write pseudocode to design a DMA-based data transfer algorithm that movesdata from an I/O device to memory while handling interrupts and minimizing CPU involvement.

START

1. Initialize the system.
2. Initialize the DMA Controller.
3. Select the I/O device.
4. Get the starting memory address.
5. Get the amount of data to be transferred.

6. Configure DMA Controller:
   Set source = I/O Device
   Set destination = Memory
   Set transfer size = Data Size Set transfer direction = I/O Device to Memory

7. Enable DMA Controller.
8. Send DMA request to the I/O device.
9. Wait until the I/O device accepts the DMA request.
10. DMA Controller requests control of the system bus.
11. CPU grants the bus to DMA Controller.
12. DMA Controller starts data transfer.

13. WHILE data transfer is not complete DO
    Read data from I/O device.
    Store data directly into memory.
    Increment memory address.
    Decrease remaining data count.
    CPU performs other tasks.
  END WHILE

14. DMA Controller releases the bus.
15. DMA Controller generates an interrupt.
16. CPU receives the DMA interrupt.
17. CPU saves its current state.
18. CPU executes DMA Interrupt Service Routine.
19. Check DMA status.

20. IF transfer was successful THEN
    Set transfer status = SUCCESS Display "DMA Transfer Completed"
  ELSE
    Set transfer status = ERROR Display "DMA Transfer Failed"
  END IF

21. Clear the DMA interrupt flag.

22. Disable DMA Controller.
23. Restore CPU state.

24. Return CPU to the previous task.

25. STOP

2. Develop pseudocode for an interrupt handling system that prioritizes vectoredinterrupts from multiple devices and dispatches appropriate service routines.

```
START

1.  Initialize the interrupt controller.
2.  Initialize all connected devices.
3.  Create an interrupt vector table.
4.  Store the service routine address for each device.
5.  Assign a priority to each device.
6.  Enable interrupts.

7.  WHILE system is running DO

        Check whether any device has generated an interrupt.

        IF no interrupt exists THEN
Continue normal CPU operation.
        ELSE            Read all pending
interrupt requests.
            Identify the devices requesting interrupts.
            Check the priority of each interrupt.
            Select the highest-priority interrupt.
            Get the interrupt vector number.
Use the vector number to find            the
corresponding service routine.

            Save the current CPU registers and status.
            Disable lower-priority interrupts if required.
            Send interrupt acknowledge signal.

            Jump to the selected Service Routine.
            Execute the Service Routine.
            Perform the required operation for the device.
            Clear the device interrupt flag.
            Send End Of Interrupt signal.

            Restore the saved CPU state.
            Re-enable interrupts.
            Return to the interrupted program.
        END IF

   END WHILE

STOP
```

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

```
START

1. Initialize the I/O system.
2. Read the required I/O operation.
3. Read the latency requirement.
4. Read the bandwidth requirement.
5. Read the amount of data to be transferred.
6. Check the type of device.

7. IF latency requirement is HIGH THEN
        Select Memory-Mapped I/O.
   ELSE IF bandwidth requirement is HIGH THEN
Select Memory-Mapped I/O.
   ELSE IF latency requirement is LOW
AND bandwidth requirement is LOW THEN
Select I/O-Mapped I/O.
   ELSE         Calculate the expected performance of both
methods.

        Compare:
            Access latency
            Data transfer rate
            CPU overhead
            Address space requirement

        IF Memory-Mapped I/O has better performance THEN
Select Memory-Mapped I/O.
        ELSE             Select
I/O-Mapped I/O.
        END IF
   END IF

8.  Configure the selected I/O method.
9.  Perform the required I/O operation.
10. Measure the actual latency.
11. Measure the actual bandwidth.

12. IF performance is below the required level THEN
        Re-evaluate the I/O method.
        Select the better method.
        Reconfigure the I/O system.
    END IF 13. Display the selected

I/O method.

STOP
```

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

```
START 1. Initialize the Bus Arbitration

System.

2. Initialize all interfaces:
      PCIe
```

```
      USB
      Thunderbolt

3. Detect all connected devices.
4. Assign a unique ID to each device.
5. Assign priority to each device.
6. Initialize the request queue.

7. WHILE the system is running DO

        Check all interfaces for communication requests.

        IF PCIe device requests communication THEN
Add PCIe request to request queue.
        END IF

        IF USB device requests communication THEN
Add USB request to request queue.
        END IF

        IF Thunderbolt device requests communication THEN
        Add Thunderbolt request to request queue.
        END IF

        IF request queue is empty THEN
Continue monitoring.
        ELSE             Check all
pending requests.
            Calculate priority for each request.

            Consider:
                Device priority
                Data size
                Urgency
                Waiting time
                Interface capability

            Select the highest-priority request.
            Grant communication access to the selected device.
            Allow the selected device to transfer data.
            Monitor the transfer.

            IF transfer is successful THEN
Mark request as completed.
            ELSE
Detect transfer error.
                IF retry is possible THEN
Retry the transfer.
                ELSE
Report transfer error.
                END IF
            END IF

            Release communication resources.
```

```
            Remove completed request from queue.
Increase waiting priority of devices
that are still waiting.         END IF

   END WHILE

STOP
```

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

```
START

1. Initialize the I/O system.
2. Detect all connected devices.

3. FOR each connected device DO

        Identify the device.
        Determine the device speed.
        Determine the amount of data to be transferred.

        IF device speed is LOW THEN

            Select POLLING mode.
            Initialize the device.

            WHILE data transfer is not complete DO
Check device status.

                IF device is ready THEN
Read data from device.
                    Store data in memory.
                    Update data count.
                ELSE                    Continue
checking device status.
                END IF
            END WHILE             Display

"Polling Transfer Completed".

        ELSE

            Select DMA mode.
            Initialize DMA Controller.

            Set source = High-Speed I/O Device.
            Set destination = Memory.
            Set transfer size.
            Set transfer direction.

            Enable DMA.
            Start DMA transfer.

            CPU performs other tasks.
```

```
            WAIT for DMA interrupt.

            IF DMA interrupt occurs THEN

                Save CPU state.
                Execute DMA Interrupt Service Routine.
                Check DMA status.

                IF DMA transfer is successful THEN
Mark transfer as completed.
                    Clear DMA interrupt.
                    Display "DMA Transfer Completed".
                ELSE
Detect transfer error.

                    IF retry is possible THEN
Restart DMA transfer.
                    ELSE
Report transfer error.
                    END IF
                END IF

                Restore CPU state.
            END IF

        END IF

   END FOR

4. Disable unused I/O resources.
5. Display final transfer status.

STOP
```